

Breve descripción del lenguaje seleccionado.

Swift, un lenguaje de programación moderno, es compilado y de múltiples paradigmas; además soporta orientación a objetos, funcionalidad, orientación a protocolos, todo esto fue creado por Apple Inc. y salió al público en el 2014.

Fue concebido para ser seguro, rápido y expresivo.

Seguro: Busca erradicar errores de programación muy frecuentes. Usa un sistema de tipos riguroso e infiere tipos para que sea más claro; también gestiona la memoria solito con el Conteo Automático de Referencias o ARC. Incorpora el rol de optionals para gestionar bien la falta de un valor, ahorrando errores de punteros nulos.

Rápido: Está optimizado para el rendimiento, se compila al código máquina nativo directamente, con el compilador LLVM; está hecho para ser casi tan veloz como C++ en muchas cosas.

Expresivo: Cuenta con una sintaxis que es limpia, corta, moderna; es sencillo de leer y de escribir. Se inspiró en lenguajes como Python, Ruby y C#.

Se emplea mayoritariamente en la creación de aplicaciones nativas, para el mundo Apple (iOS, macOS, iPadOS, watchOS, tvOS) a pesar de ello su uso en el desarrollo del lado del servidor (server-side Swift) está incrementando.

i. Lista y explique las estructuras de control de flujo presentes.

Swift nos ofrece un compendio total de estructuras de control de flujo, muy parecidas a las de C, pero con mejoras considerables en seguridad y potencia.

Condicionales (Bifurcación)

if / else if / else

Se activan bloques de código según una condición booleana, ¿cierto o falso? No exige paréntesis alrededor de la condición (por ejemplo si $x > 10$) aunque sí se necesita llaves ({}) para el bloque de código, incluso en líneas sencillas.

switch

En mi opinión el switch de Swift es superior al de C. Compara un valor con un montón de patrones.

Exhaustivo: Tiene que cubrir todos los casos posibles del valor, sino se deben considerar un caso default.

Sin "caída" (No fallthrough) por defecto: Contrario a C, la ejecución no avanza al siguiente caso. Si esto se desea, hay que utilizar la palabra clave fallthrough, claramente.

El guard

Es una estructura tipo "salida rápida". Se utiliza para asegurar que una condición tiene que ser verdadera para que el código siga adelante.

Siempre requiere un bloque else que debe salir del ámbito actual (usando return, break, continue, o throw).

Se usa muchísimo para validar entradas de funciones o desenvolver optionals sin problemas (ej. `guard let name = optionalName else { return }`).

Bucles (Iteración)

for-in

Es el bucle principal en Swift. Itera sobre una secuencia, como un rango (`for i in 1..5`), un array (`for item in items`) o un diccionario (`for (key, value) in dict`).

while

Evalúa una condición antes de cada iteración. El bucle se repite siempre y cuando la condición sea verdadera. Si la condición es falsa al principio, el bloque nunca se ejecuta.

repeat-while

Equivalente al do-while en otros lenguajes. Ejecuta el bloque de código una vez, y luego evalúa la condición después de la iteración. Se repite mientras la condición es verdadera.

Control de Transferencia (Saltos)

break

Sale, de inmediato, de un bucle (for while repeat-while) o de un bloque switch.

continue

Interrumpe la iteración presente del bucle, y avanza a el principio de la iteración siguiente.

return

Sale de una función o método; opcionalmente retorna un valor.

throw

Se utiliza para lanzar (señalar) un error en una función declarada como throws. El control cambia al bloque catch correspondiente.

fallthrough

Empleado dentro de un switch para forzar que la ejecución siga en el próximo caso (case), imitando el comportamiento predeterminado de C.

ii. Diga cómo se evalúan las expresiones y funciones.

A. ¿Utiliza evaluación normal o aplicativa? ¿Usa evaluación perezosa?

Evaluación Aplicativa (por defecto)

Swift emplea la evaluación aplicativa (también llamada eager evaluation o evaluación ansiosa) por defecto.

Significa esto que, al llamar una función, todos los argumentos (expresiones) suministrados a esa función se evalúan por entero antes de entrar en el cuerpo de la función.

Bueno, con `miFuncion(a(), b())`, primero se examina `a()`, después se evalúa `b()`, y ya ahí recién se llama a `miFuncion` usando los resultados de `a` y `b`.

Evaluación Perezosa (Disponible)

A pesar de que usualmente se aplica un comportamiento aplicativo, Swift da herramientas explícitas para la evaluación perezosa (lazy evaluation):

Propiedades lazy var: Una variable de instancia que se declara como lazy var, no computará su valor inicial hasta que se accede por primera vez.

Colecciones lazy: Las colecciones, como Arrays, Dictionaries, etc. tienen una propiedad `.lazy` ej `miArray.lazy.map { }`. Esto hace una vista perezosa en la que las operaciones tipo map, filter no se ejecutan hasta que los elementos se necesitan (por ejemplo, en un bucle for-in que siga).

@autoclosure: Este atributo, que se usa en parámetros de funciones, convierte automáticamente un argumento en un closure (una función anónima). La expresión no se evalúa al llamar la función, sino que la función decide cuándo (y si) se ejecuta el closure para obtener el valor.

Ejemplo importante: Los operadores de cortocircuito `&&` y `||` y el operador Nil-Coalescing `?` usan esto. En `a || b()`, `b()` nunca se evalúa si `a` es verdadero.

La valoración de los argumentos y operandos procede de izquierda a derecha, de derecha a izquierda, o quien sabe, en un orden al azar.

B. La evaluación de argumentos/operandos se hace de izquierda a derecha, de derecha a izquierda o en un orden arbitrario.

De Izquierda a Derecha (Comúnmente)

Swift posee reglas de evaluación bastante más rigurosas y predecibles, en comparación a C o C++.

Argumentos de funciones; la evaluación de los argumentos, cuando se llama a una función, siempre ocurre de izquierda a derecha.

En `miFuncion(a(), b(), c())`, `a()` se evalúa primero, seguido por `b()` y, por último, `c()`.

Operandos de operadores: Con la mayoría de los operadores binarios, como `+`, `&&`; el operando de la izquierda se valora primero, y luego el de la derecha.

En `a() + b()`, `a()` es la prioridad, y entonces viene `b()`.

Una Excepción: Operadores de Asignación.

La mayor excepción es la asignación (=) y, además, los operadores de asignación compuestos (tipo +=, =).

En la asignación el lado derecho (RHS) es el primero en ser valorado, antes que el lado izquierdo (LHS)

Por ejemplo; En `miArray[indice()] = valor()`, lo primero es `valor()`, luego `indice()`, después `miArray`, y se asigna al índice.